2026 IEEE International Conference for Convergence in Computing Technology (I3CTCON)
Lonavala, India. Mar 14 - 15, 2026

2026 IEEE International Conference for Convergence in Computing Technology (I3CTCON) | 979-8-3315-7656-1/26/$31.00 ©2026 IEEE | DOI: 10.1109/I3CTCON68242.2026.11508116

Evaluating Machine Learning Techniques on Big Data Frameworks for MQTT IoT Security

Geda Tejesh Chowdary, Paramkusam Sriharsha, Yelipe Gowtham, Niharika Panda
Department of Computer Science and Engineering
Amrita School of Computing, Bengaluru
Amrita Vishwa Vidyapeetham, India
bl.en.u4cse22114@bl.students.amrita.edu, bl.en.u4cse22141@bl.students.amrita.edu,
bl.en.u4cse22163@bl.students.amrita.edu, p_niharika@blr.amrita.edu

***Abstract*—Message Queuing Telemetry Transport (MQTT) has grown to become a market leader in the IoT communication protocol due to its lightweight publish-subscribe design, which is suitable for constrained devices and real-time sensor communication. Nevertheless, MQTT in its nature does not have well-developed and embedded security features, and it is exposed to cyberattacks targeting brokers, clients, and flows of messages. The MQTTEEB-D dataset illustrates practical attacks, i.e., Denial of Service (DoS), Slow DoS against Internet of Things Environments (SlowITe), malformed data injection, brute-force authentication, and publish flooding, which are executed on a live Internet of Things (IoT) testbed of Raspberry Pi brokers and MySignals sensors. These attacks take advantage of protocol flags, timing behavior, message sizes, and connection parameters, and display vulnerabilities that are not easily identified using traditional rule-based security systems. Consequently, the identification of subtle anomalies in MQTT traffic by machine learning-based Intrusion Detection System (IDS) solutions is evident so that it is adequate. The project targets the creation of ML classification models that are trained on actual MQTTEEB-D data to differentiate between normal and malicious MQTT patterns with a high degree of accuracy to increase the security and resilience of IoT networks.**

***Index Terms*—MQTT, IoT security, Intrusion detection system, Machine Learning, Network traffic, Real time attack execution.**

I. Introduction

The constantly growing, globally interconnected world has individuals, machines and computerized services in constant communication resulting in huge amounts of network traffic that should be safe and reliable. The scale of communication is increasing among smart devices, cloud services, and IoT networks, and with it, the number of attack vectors, which can be exploited by cyberattacks to propagate quickly and interrupt critical services, is increasing. Real-time operations are often vulnerable to service attacks like DoS floods, brute force, data corruption and protocol misuse which often targets communication channels. Based on the documentation of MQTTEEB-D database, cyberattacks may negatively impact network performance, overload communication layers, or impact the behavior of the system, particularly when constant access to data is essential to healthcare, smart home automation, and industrial monitoring. Thus, Intrusion Detection Systems (IDS) have become an absolute necessity. They read, interpret, and prevent malicious acts of traffic and ensure digital communications are withholding proper protection. IDS solutions guarantee availability, reliability, and trust in the modern world of high connectivity in the digital environment, where secure communication is the most significant requirement among people as well as devices.

MQTT has grown to become a market leader in the IoT communication protocol due to its lightweight publish-subscribe design, which is suitable in constrained devices and real-time sensor communication. Nevertheless, MQTT in its nature does not have well-developed and embedded security features, and it is exposed to cyberattacks, targeting brokers, clients, and flows of messages. The MQTTEEB-D dataset illustrates practical attacks i.e. DoS, SlowITe, malformed data injection, brute-force authentication, publish flooding, which are executed on a live IoT testbed of Raspberry Pi brokers and MySignals sensors. These attacks take advantage of protocol flags, timing behavior, message sizes and connection parameters and display vulnerabilities that are not easily identified using traditional rule-based security systems. Consequently, the identification of the subtle anomalies in MQTT traffic by machine learning-based IDS solutions so that it is adequate is evident. The project targets the creation of ML classification models that are trained on actual MQTTEEB-D data to differentiate between normal and malicious MQTT patterns with a high degree of accuracy to increase the security and resilience of IoT networks.

- The paper builds a scalable PySpark intrusion detection pipeline that effectively analyzes the real-world MQTEEB-D dataset to analyze MQTT security.
- Compare and contrast several machine learning models, which indicate that Gradient-Boosted Trees (GBT) yield better detection results concerning all attack types.
- Conduct a lot of hyperparameter optimization of Decision Tree and GBT models, with a great improvement in accuracy and creating an extremely reliable classifier in MQTT-based IoT networks.

The subsequent sections of this paper are as follows: Section II provides the related work. Section III discusses the methodology and implementation. Section IV provides the results. Section V concludes the paper with future work.

Authorized licensed use limited to: University of Malaya. Downloaded on August 25,2026 at 06:49:38 UTC from IEEE Xplore. Restrictions apply.

II. Related Works

Aqachtoul et al. [1] present MQTEEBD, which is a real-world MQTT cybersecurity dataset created with an IoT testbed. It involves their work, which consists of full exploratory data analysis and gives pre-processed version of datasets, which allows further machine-learning and deep-learning in the field of intrusion detection studies on MQTT driven IoT systems. Tanveer et al. [2] introduce LightEnsemble-Guard which is a lightweight ensemble of LightGBM, XGBoost and Extra Trees that predict IoT attacks in real-time. Their model deals with unbalanced data, minimizes computation costs, and their model has strong performance on five-fold validation and ROC-based evaluation measures.

In Nazir et al. [3], the researchers empirically compare ensemble learning and hybrid CNNLSTM networks using heterogeneous IoT security datasets. Their results indicate that hybrid deep-learning models significantly outperform the traditional machine-learning solutions in the process of detection of complex IoT threats, especially on large datasets like IoT-23 and CICIDS2017. Salama et al. [4] combine machine-learning, MQTT, and Blockchain to provide better security of the IoT systems. Their paper looks at supervised, unsupervised, and reinforcement learning models in blockchain-based systems and show that they are better refined in terms of communication integrity with the help of smart contracts and MQTT publishsubscribe protocols.

Putro et al. [5] propose a method of feature engineering based on the Euclidean distance which enhances the intrusion detection in MQTT networks. They benchmark a variety of ML algorithms on the RT-IoT2022 dataset and point out that there is a lack of diversity of the datasets and the complexity of bi-directional flow features. Solanki et al. [6] suggest a stacked version of ensemble IDS made of Gaussian Naive Bayes, SVM and MLP plus a logistic regression meta model. Multi-head attention is used in their system, and the Uni-Flow, Bi-Flow, and Packet-Flow of MQTT-IoT-IDS2020 are evaluated to enhance the accuracy of detecting the anomalies.

Jiwangkurra et al. [7] introduce an MQTT classification structure that deals with platforms, databases, energy, and industrial realms. Their overview of IoT deployments points to the energy-efficient nature of MQTT and MQTT-SN especially with constrained IoT devices that need a minimum of CPU usage and minimal message processing. Dikii et al. [8] examine detection of DoS attacks in MQTT networks based on Multilayer Perceptron, Random Forest, and SVM classification. They processed feature vectors of extracted MQTT with a modular framework and used simulated data obtained using Raspberry Pi testbeds and using Paho-MQTT traffic sources.

Sharma et al. [9] compare MQTT-based attacks by IoT and the use of SVM, Random Forest, AdaBoost, and XG-Boost. They use the dataset UFPI_NCAD_IOT_ATTACKS and proceed with a full ML workflow, which consists of preprocessing, feature extraction, and traffic-level analysis to classify various kinds of attacks, including DoS and MITM. Lakshminarayana et al. [10] discuss the vulnerabilities of MQTT-layer and analyze detection schemes such as ML-based IDS, rule-based systems, fuzzers and authentication systems. They find flaws in the existing MQTT datasets, and highlight how lightweight and scalable machine-learning based intrusion detection solutions are required. Kadam et al. [11]highlighted PySpark's scalability for handling large multi-city housing datasets and improving price prediction accuracy. Sharma et al. [12] developed a CNN-based plant disease detection system using PySpark for distributed processing and Tkinter for an intuitive farmer-facing interface, achieving high diagnostic accuracy. Pranave et al. [13] presents a fast keyword-based desktop search that reads file contents once and then retrieves relevant documents almost instantly. Panda et al. [14] studies blackhole attacks in RPL-based LLNs under static and mobile settings, showing their impact on PDR, latency, and other performance metrics. Niharika et al. [15] proposes an SVM-based attack detection scheme for RPL/6LoWPAN IoT networks, achieving about 84% prediction accuracy.

The reviewed literature also reveals a great development of MQTT-based IoT security on the basis of machine-learning, deep-learning, and ensemble-based intrusion detection models. Research provides actual MQTT data, weight-optimal lightweight ensembles, hybrid CNN-LSTM networks and feature engineering using the Euclidean distance to improve the quality of threat detection. Together, the articles point to the ongoing issues of limited availability of datasets, adapting attack patterns, and the demand to have scalable, low-overhead and real-time, IDS software that is applicable to limited IoT models.

III. Methodology and Implementation

A. System Requirements

The suggested intrusion detection system will need a computing environment with the capability to process large datasets of network data and conduct distributed machine-learning processes effectively. To handle both the execution of PySpark, feature engineering and the training of classification models on data sets that are larger than one million records, the system would require at least 8 GB RAM and a multi-core processor to run. It is proposed that a machine with 20 GB storage be used to store raw data, processed data, and generated model outputs. The Python 3.x software stack will be used, together with Apache Spark and the required ML libraries, including PySpark MLlib, and visualization packages, including Matplotlib and Seaborn. The system can also support the use of either a GPU to train heavy models, like Gradient Boosted Trees. This design allows the process of preprocessing, training of the model, evaluation of the MQTT intrusion detection pipeline to be executed smoothly.

B. Dataset Description

The data comprises packet-level network traffic logs of MQTT communication over TCP as in Table I. A row denotes one packet and contains timing characteristics, TCP control

Authorized licensed use limited to: University of Malaya. Downloaded on August 25,2026 at 06:49:38 UTC from IEEE Xplore. Restrictions apply.

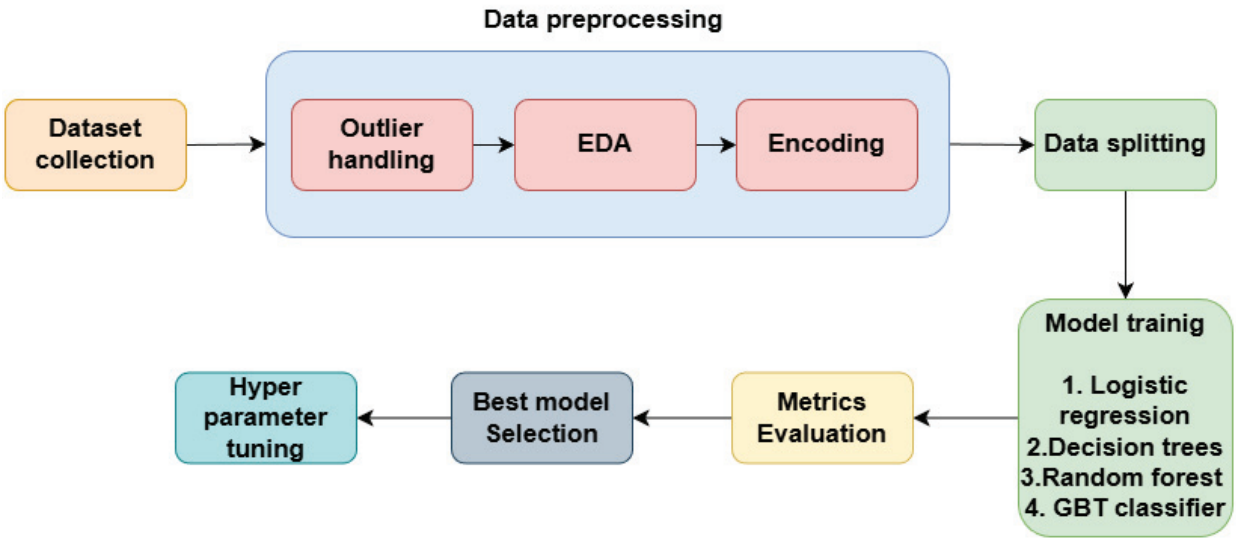


Fig. 1. System Architecture Diagram

data and MQTT-specific elements like connection flags, duplicate flags, keep-alive values, QoS values, and message content attributes. These characteristics define the behavior of a transport layer and the message patterns of the application layer. The data also has a target label of whether a given traffic is normal or malicious. The presented structure renders the dataset applicable in machine learning-based intrusion detection, anomaly analysis, and acquisition of the MQTT traffic behavior in IoT settings.

TABLE I
KEY FEATURES IN MQTTEEB-D DATASET

No	Name	Protocol Layer	Data Type
1	timestamp	TCP	object
2	tcp_flags	TCP	int64
3	tcp_time_delta	TCP	object
4	tcp_len	TCP	int64
5	mqtt_conack_flags	MQTT	object
6	mqtt_conflag_cleansess	MQTT	int64
7	mqtt_conflags	MQTT	object
8	mqtt_dupflag	MQTT	int64
9	mqtt_hdrflags	MQTT	int64
10	mqtt_kalive	MQTT	int64
11	mqtt_msg	MQTT	object
12	mqtt_qos	MQTT	int64
13	target	Classification	float64

C. System Architecture

The proposed scalable machine-learning pipeline as shown in Fig. 1 has the system architecture starting with the collection of data in which MQTT traffic logs would be aggregated by different sources of IoT communications. This raw information is subjected to a structured data preprocessing process, which consists of outlier processing, exploratory data analysis (EDA), and coding. Removal of outliers means that extreme or noisy data do not affect the training process in any way, whereas EDA gives some statistical understanding of the feature distributions and correlations. The indexing and encoding operations are used to express the categorical variables like MQTT flags in the form of machine readable numeric values. The preprocessed data is then subjected to train-test split, which allows the system to assess by its own the capacity of the classification models to generalise.

The second step involves the training of the model in which four supervised machine-learning models, which include the Logistic Regression, Decision Trees, Random Forest, and Gradient-Boosted Trees, are being trained using the PySpark MLlib. Patterns are learnt in each model based on the prepared feature vector of the characteristics of MQTT packets. The system then undertakes metric assessment where accuracy, precision, recall and F1-score, and confusion matrices are calculated in order to test the performance of the classification. According to these findings, the architecture has a best-model selection step, which finds the model that behaves best on unseen data. Lastly, the chosen model is hyperparameter-tuned to improve performance optimally with the use of grid search or cross-validation, so that the system can be highly reliable and scalable to detect intrusion into the IoT.

D. Machine Learning Models

This work combines four managed machine-learning models: Logistic Regression, Decision Trees, Randome Forest and Gradient Boosted Trees which have been performed using PySpark MLlib. All models are trained to identify patterns in encoded MQTT features to determine legitimate or attack network traffic with scalable distributed training.

1) Logistic Regression (LR): The base model of this system is Logistic Regression which is a linear classification model. It approximates the likelihood of an MQTT packet as a part of a specific attack category via a weighted linear sum of the feature set. As MQTT operations such as TCP flags, time-delta values, and connection parameters can be transformed

Authorized licensed use limited to: University of Malaya. Downloaded on August 25,2026 at 06:49:38 UTC from IEEE Xplore. Restrictions apply.

into numerical columns, LR can effectively learn decision limits via a gradient descent efficiently. LR is computationally lightweight, can be trained on large data, and performs well in the situations when the classes are linearly separable or feature interactions are not significant. The model used in the PySpark pipeline takes the constructed feature vector as its input and gives class predictions on all MQTT flows in the MQTTEEB-D dataset. It is easy and fast to use, hence making it a vital starting point of assessing performance gains of tree-based models.

2) Decision Tree Classifier (DT): A Decision Tree classifier is a hierarchical model formed of a series of rules formed by repeated subdivision of the data according to feature values forming a tree-shaped representation. The protocol-level indicators, mqtt_qos, connection flags, and mqtt_qos, provide protocol-level rules to the model that learn interpretable rules in the context of MQTT intrusion detection. The internal nodes are depicting decision based on features, and the leaf nodes depict the predicted attack type. DTs can be useful in determining which MQTT attributes have the largest impact on the classification, which is advantageous given that such attributes should be considered to provide transparency and readability. Nevertheless, single trees are vulnerable to overfitting complex datasets such as MQTTEEB-D, which are mixed attack patterns. However, the Decision Trees are the basis of more sophisticated ensemble algorithms including Random Forest and GBT, which is a major building block in the architecture.

3) Random Forest Classifier (RF): Random Forest is a technique of ensemble learning which constructs several Decision Trees using various subsets of training data. RF enhances robustness in this system by lessening overfitting and generalizing different MQTT attacks including DoS, SlowITe, brute force, flooding, and malformed packet injection. The various trees each portray varying relationships between MQTT fields and the model is an amalgamation of conclusions made by all the trees together generating a final class prediction. PySpark MLlib is distributed across worker nodes to build trees, and hence enables the forest to scale comparatively in excess of a million feature-processed MQTT samples. Random Forest suits structured intrusion detection activities, where non-linear patterns, feature interactions and imbalanced distributions of traffic are quite well addressed.

4) Gradient-Boosted Trees (GBT Classifier): Gradient-Boosted Trees create an ensemble in sequential order with a tree succeeding earlier trees, correcting their errors. This renders GBT an efficient non-linear classifier that can detect small anomalies in MQTT behavior, e.g. anomalous timing variation, header flags manipulation, or control messages that are irregular. The process of boosting promotes sensitivity to less common attack signatures, which is necessary in the context of IoT where some forms of attacks are less common. It is an ensemble of numerous error-correcting trees, which are shallow and implemented with PySpark as the distributed GBTClassifier and extended with One-Vs-Rest in multi-class classification, producing a model with high predictive capability. GBT is normally the strongest model out of the four as it can learn intricate decision boundaries, as illustrated in the system architecture, since it is capable of learning the complex decision boundaries as a result of the MQTT feature space.

E. Evaluation Metrics

1) Accuracy: Accuracy is the most common measure that determines the correctness of the model in general by determining the fraction of the correctly predicted samples.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

2) Precision: Precision represents the fraction of cases when the model correctly predicts attacks among all the cases that the model predicts to be attacks.

$$\text{Precision} = \frac{TP}{TP + FP}$$

3) Recall: The Recall (or Sensitivity) is used to determine whether the model can properly detect all real cases of attacks, and there are few false negatives.

$$\text{Recall} = \frac{TP}{TP + FN}$$

4) F1-Score: F1-Score offers a compromise between accuracy and recall, which is a single measure of performance that can be applied to unbalanced data such as network data.

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

The combination of these measures can provide a detailed assessment of how well the system can identify attacks.

IV. Results and discussion

The results section is divided into two sections, they are the baseline models analysis and fine tuned model analysis, the results are discussed below.

A. Base Machine Learning Model Analysis

The baseline machine learning models, which included LR, RF, DT, and GBT, were tested in terms of accuracy, precision, recall, and F1-score. The results of the comparisons are found in Table II where one can see clearly the difference in capabilities of the models.

TABLE II
Performance Comparison of Machine Learning Models

Model	Accuracy	Precision	Recall	F1-Score
LR	0.5750	0.5397	0.5750	0.5188
RF	0.8760	0.8864	0.8760	0.8577
DT	0.8807	0.8862	0.8808	0.8674
GBT	**0.9170**	**0.9159**	**0.9170**	**0.9122**

Logistic Regression has the lowest performance with the accuracy of 0.5750 and an F1-score of 0.5188. Despite its confusion matrix, LR erroneously classifies a number of attack types especially SlowITe, brute force and malformed packets, which suggests that LR is not able to capture non-linear associations in MQTT traffic.

Authorized licensed use limited to: University of Malaya. Downloaded on August 25,2026 at 06:49:38 UTC from IEEE Xplore. Restrictions apply.

However, tree-based models have much better performance. The Decision Tree classifier has the best accuracy of 0.8807, successfully learning a good number of protocol-level patterns but falsely classifying attacks with similar behavior moderately. Random Forest also improves the classification stability by minimizing the variance between trees, with an accuracy of 0.8760, and showing a better resilience to various attacks.

The GBT classifier is the most effective of all models because it has the highest accuracy 0.9170) and the highest precision 0.9159), recall 0.9170), and F1-score 0.9122). The GBT confusion matrix demonstrates low levels of false positives and false negatives of all six classes, which proves its effectiveness in identifying subtle MQTT anomalies and a combination of complex features better than LR, DT, or RF. This causes GBT to be the most dependable base model in detecting intrusion in an MQTT-based IoT setting. The accuracy bar plot of LR, RF, DT and GBT can be seen in Fig. 2, where we can analyze that GBT is the best model.

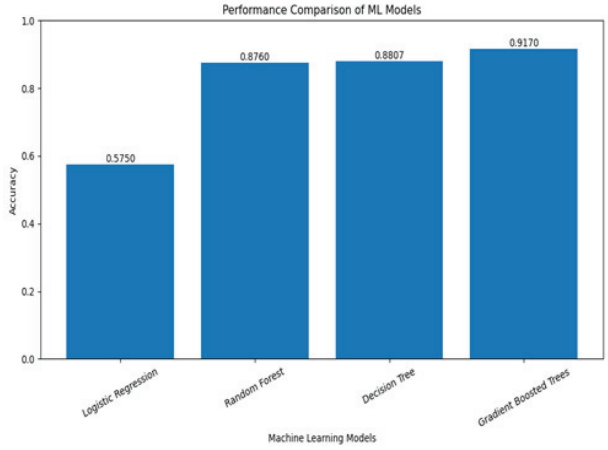


Fig. 2. Performance Bar Plot of Base Models

B. Fine Tuned Machine Learning Model Analysis

1) Fine Tuning Decision Tree: In order to enhance the original performance of the Decision Tree classifier, hyperparameter optimization was performed by adjusting two important parameters which included maxDepth and maxBins. The first one (maxDepth = 10, maxBins = 64) was taken as a reference point. Table III demonstrates that, with a rise in model complexity, it was possible to observe a statistically significant improvement in all evaluation metrics.

TABLE III
DECISION TREE HYPERPARAMETER TUNING RESULTS

Hyperparameters	Accuracy	Precision	Recall	F1-Score
maxDepth = 15, maxBins = 128	0.8989	0.8995	0.8990	0.8911
maxDepth = 20, maxBins = 256	**0.9191**	**0.9189**	**0.9190**	**0.9144**

The initial tuned configuration (maxDepth = 15, maxBins = 128) provided better results, with the highest accuracy of 0.8989, the highest precision of 0.8995, the highest recall of 0.8990 and the highest F1-score of 0.8911. This shows that with more permissible splits, the tree was able to identify more decision boundaries of MQTT traffic patterns.

The second tuning parameter (maxDepth = 20, maxBins = 256) had the highest results as the accuracy is 0.9191, precision is 0.9189, recall is 0.9190, and F1-score is 0.9144. Such gains were also evident in the confusion matrices of the two tuning configurations where Setup-2 had fewer false negatives and better separation between similar classes of attacks.

In general, the tuning experiments prove that more model depth, more bin granularity improves the capability of the Decision Tree to detect more difficult MQTT anomalies, making the model much more effective as an intrusion detection process. The bar plot of tuned DT is shown in Fig. 3. The confusion matrix for the best tuned DT i.e., version3(maxDepth=20) can be seen in Fig. 4 respectively.

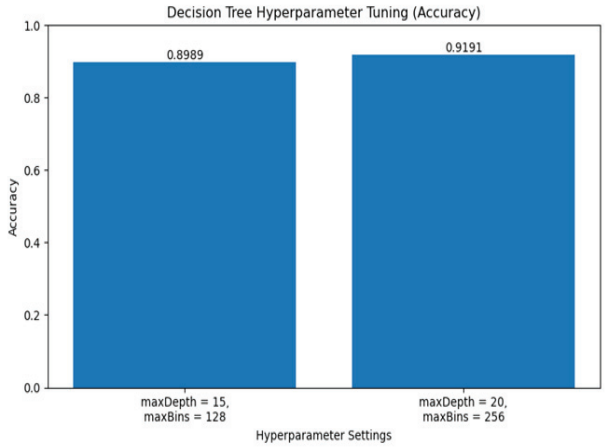


Fig. 3. Performance Bar Plot of Tuned DT

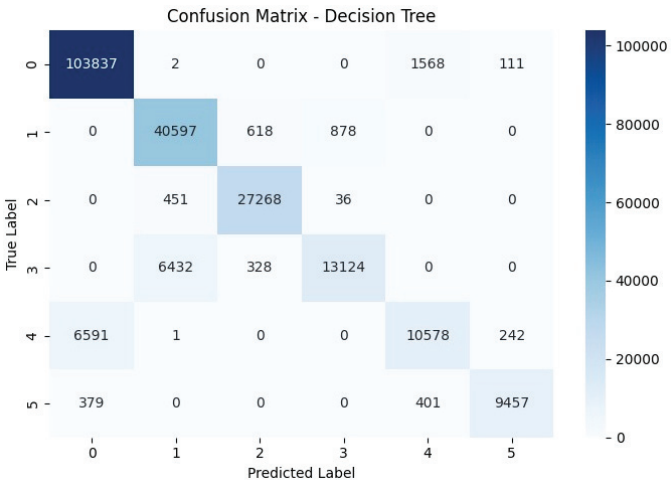


Fig. 4. Version 3 of Decision Tree Confusion Matrix

Authorized licensed use limited to: University of Malaya. Downloaded on August 25,2026 at 06:49:38 UTC from IEEE Xplore. Restrictions apply.

2) Fine Tuning Gradient Boosting Trees: In order to enhance the performance of GBT classifier, various hyperparameter settings were tested by modifying the depth of the model, bin size, boosting series and learning rate. The first parameter (maxDepth = 8, maxBins = 50, maxIter = 50, stepsize = 0.1) was used as the control. Table IV demonstrates the results of the tuning of the model as the model parameters were optimized; it is evident that the values of all metrics improved.

TABLE IV
GBT HYPERPARAMETER TUNING RESULTS

Hyperparameters	Accuracy	Precision	Recall	F1-Score
Depth = 8, Bins = 50, maxIter = 100	0.9230	0.9231	0.9230	0.9180
Depth = 8, Bins = 50, maxIter = 150	0.9318	0.9315	0.9317	0.9282
Depth = 12, Bins = 80, maxIter = 50, stepSize = 0.15	**0.9534**	**0.9527**	**0.9534**	**0.9521**

The initial tuning configuration enhanced boosting iterations to 100 with a slight improvement in performance with an accuracy of 0.9230 and F1-score of 0.9180. A further increase in the number of the iteration to 150 increased the accuracy to 0.9318 which shows a more accurate learning of attack patterns in the six MQTT traffic classes.

The best result was obtained using the third setting (maxDepth = 12, maxBins = 80, maxIter = 50, stepsize = 0.15) and had the highest accuracy (0.9534), precision (0.9527), recall (0.9534), and F1-score (0.9521). The performance in confusion matrix was also better in this version with fewer false positives and false negatives between all the attack categories.

These findings indicate that more profound trees and an incrementally higher learning rate allow the GBT model to discover the more intricate non-linear decision boundaries hence it is the most potent general classifier of the MQTTEEB-D dataset. The bar plot of tuned GBT is shown in Fig. 5. The confusion matrices of Version 2 (maxIter=100), Version 3 (maxIter=150) and Version 4 (maxIter=50) can be seen in Fig. 6, 7 and 8 respectively.

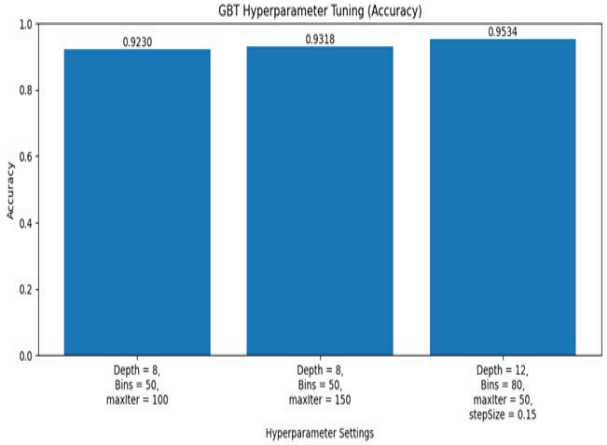


Fig. 5. Performance Bar Plot of Tuned GBT

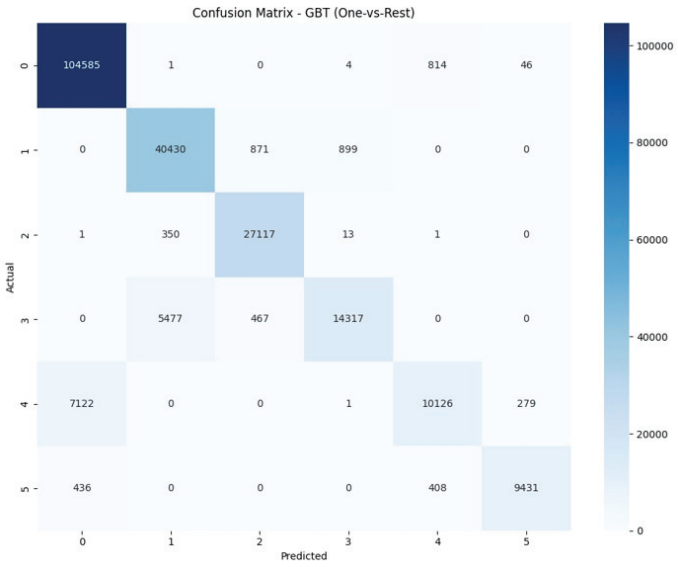


Fig. 6. Version 2 of Gradient Boosting Trees Confusion Matrix

V. CONCLUSION AND FUTURE WORK

The experiments presented on the MQTTEEB-D data indicate that machine learning models are able to effectively detect MQTT-based cyberattacks when implemented on a scalable big-data pipeline such as PySpark. Decision Tree and GBT were the best of the baseline models considered, and inspired further experimentation and tuning. Using progressive hyperparameter optimization, the two models achieved significant improvements in terms of accuracy, precision, recall and F1-score. GBT, specifically, demonstrated the most consistent and better performance, and its last Version 4 had nearly 95% accuracy, becoming the most dependable classifier to work in real-time intrusion detection in MQTT-based IoT settings. The findings are a clear indication of the power of boosting-based ensembles in capturing fine-grained behavioral differences between multiple categories of attacks and low false-positive and false-negative rates. An extension to future work is that this study can be experimented with stacking or blending methods using multiple classifiers to further increase the predictive performance. Also, investigating the deep learning design structures, including LSTMs, CNN-LSTM hybrids, or Transformers might allow the system to learn temporal and contextual relationships in the MQTT traffic. Lastly, the integration of Explainable AI (XAI) systems such as SHAP and LIME would enhance transparency, which would allow security analysts to interpret the significance of the feature and comprehend the decision the model has made when determining an intrusion during the intrusion detection process.

REFERENCES

[1] A. Aqachtoul, K. Karam, A. Elamrani, M. Najib, N. Rafalia, and M. Bakhouya, "Mqtteeb-d: A real-world iot cybersecurity dataset for

Authorized licensed use limited to: University of Malaya. Downloaded on August 25,2026 at 06:49:38 UTC from IEEE Xplore. Restrictions apply.

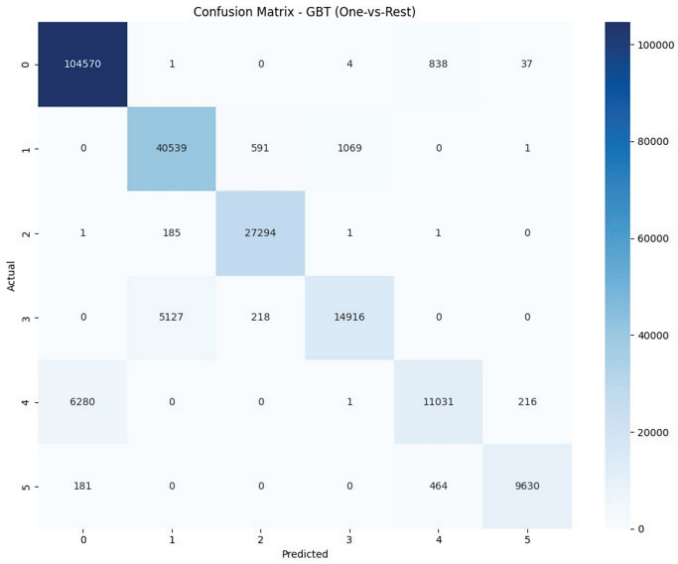


Fig. 7. Version 3 of Gradient Boosting Trees Confusion Matrix

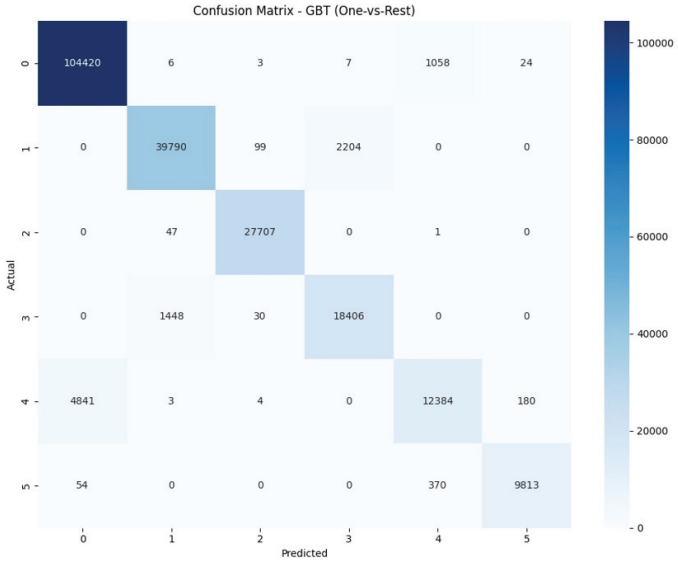


Fig. 8. Version 4 of Gradient Boosting Trees Confusion Matrix

ai-powered threat detection in mqtt networks," *Data in Brief*, p. 111897, 2025.

[2] M. U. Tanveer, K. Munir, M. Amjad, H. J. Alyamani, A. Bermak, and A. U. Rehman, "Lightensemble-guard: An optimized ensemble learning framework for securing resource-constrained iot systems," *IEEE Access*, 2025.

[3] A. Nazir, J. He, N. Zhu, A. Wajahat, F. Ullah, S. Qureshi, and M. S. Pathan, "Empirical evaluation of ensemble learning and hybrid cnn-lstm for iot threat detection on heterogeneous datasets," *The Journal of Supercomputing*, vol. 81, no. 6, p. 775, 2025.

[4] M. Salama, "Integrating blockchain, mqtt, and machine learning for enhanced iot applications: A comprehensive survey," *Sakarya University Journal of Computer and Information Sciences*, vol. 8, no. 3, pp. 570–591, 2025.

[5] I. H. Putro, T. Ahmad, and R. M. Ijtihadie, "Enhancing mqtt intrusion detection in iot using machine learning and feature engineering," *IEEE Open Journal of the Communications Society*, 2025.

[6] M. Solanki and S. Gupta, "A novel intrusion detection framework using ensemble learning in mqtt iot applications," *Annals of Mathematics and Artificial Intelligence*, pp. 1–23, 2025.

[7] S. Jiwangkura, P. Sophatsathit, and A. Chandrachai, "Mqtt of iot classification in energy saving," in *proceeding of 11th International Conference on Data Mining, Computers, Communication and Industrial Applications (DMCCIA-2017), Kuala Lumpur*, 2017.

[8] D. Dikii, S. Arustamov, and A. Grishentsev, "Dos attacks detection in mqtt networks," *Indonesian Journal of Electrical Engineering and Computer Science*, vol. 21, no. 1, pp. 601–608, 2021.

[9] A. Sharma and H. Babbar, "Towards resilient iot security: An analysis and classification of attacks in mqtt-based networks," in *2024 2nd International Conference on Advancement in Computation & Computer Technologies (InCACCT)*. IEEE, 2024, pp. 122–125.

[10] S. Lakshminarayana, A. Praseed, and P. S. Thilagam, "Securing the iot application layer from an mqtt protocol perspective: Challenges and research prospects," *IEEE Communications Surveys & Tutorials*, vol. 26, no. 4, pp. 2510–2546, 2024.

[11] K. P. Dharmaraj, P. K. Gupta, K. Ajith, M. Kolli, S. Khare, and N. Panda, "Real estate price prediction using pyspark mllib," in *2024 IEEE International Conference on Smart Power Control and Renewable Energy (ICSPCRE)*, 2024, pp. 1–6.

[12] V. Sharma, S. Kannan, S. Tanya, and N. Panda, "Detecting plant diseases at scale: A distributed cnn approach with pyspark and hadoop," *Procedia Computer Science*, vol. 235, pp. 1044–1057, 2024, international Conference on Machine Learning and Data Engineering (ICMLDE 2023). [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1877050924007750

[13] P. K.C., S. S. S, V. K. Reddy Challa, and N. Panda, "System search service implementation based on a custom lexical search," *Procedia Computer Science*, vol. 235, pp. 1548–1557, 2024, international Conference on Machine Learning and Data Engineering (ICMLDE 2023). [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1877050924008226

[14] N. Panda and M. Supriya, "Blackhole attack impact analysis on low power lossy networks," in *2022 IEEE 3rd Global Conference for Advancement in Technology (GCAT)*, 2022, pp. 1–5.

[15] N. Panda and M. Supriya, "Blackhole attack prediction in wireless sensor networks using support vector machine," in *Advances in Signal Processing, Embedded Systems and IoT*, V. V. S. S. S. Chakravarthy, V. Bhateja, W. Flores Fuentes, J. Anguera, and K. P. Vasavi, Eds. Singapore: Springer Nature Singapore, 2023, pp. 321–331.

Authorized licensed use limited to: University of Malaya. Downloaded on August 25,2026 at 06:49:38 UTC from IEEE Xplore. Restrictions apply.